VIT BHOPAL

TOPIC: Meeting Scheduler with Reminders

Submitted to:
Dr. D.Saravanan
Course Code: CSE1021
Slot: B11+B12+B13

Submitted By:
Tanuj Singh
25BCE11140

vityarthi
YOUR LEARNING DESTINATION

2. Introduction

The Meeting Scheduler is a lightweight, console-based application developed in Python that allows users to add, view, delete, and get reminders for upcoming meetings. All data is persistently stored in a plain text file (bucket.txt).

Objectives

- **Provide a simple interface for managing meetings**
- **Automatically remind users of meetings within the next 24 hours**
- **Persist data between program runs**
- **Demonstrate file handling, datetime operations, and basic CRUD functionality**

Technologies Used: Python 3, Standard Library (datetime, time)

---

3. Problem Statement

In day-to-day life, people often forget important meetings or appointments due to lack of reminders. Existing calendar apps are feature-heavy and require internet or mobile devices. This project provides a minimal, offline, console-based reminder system that runs on any machine with Python, stores meetings locally, and actively notifies users of meetings scheduled in the next 24 hours.

---

4. Functional Requirements

ID	Requirement
01	Add a new meeting with title, date, and time
02	View all meetings sorted by date/time
03	Delete an existing meeting
04	Show upcoming meetings within next 24 hours
05	Load/save meetings from/to a file automatically
06	Exit the application cleanly

---

5. Non-functional Requirements

ID	Category	Requirement
01	Usability	Simple numbered menu, clear prompts
02	Reliability	Data persisted even after program closes
03	Performance	Instant response (< 0.5 sec) for all operations
04	Portability	Runs on any OS with Python 3 installed
05	Maintainability	Clean, modular, well-commented code

---

6. System Architecture

- **Single-tier console application**
- **Language: Python 3.x**
- **Data Storage: Plain text file (bucket.txt) – one meeting per line**
- **Format: Title,YYYY-MM-DD HH:MM**
- **No external dependencies → highly portable**

Technology Stack
Frontend/Logic: Python (console)
Persistence: Text file (bucket.txt)

Modules Used: datetime, time

---

7. Design Diagrams

7.1 Use Case Diagram

Actors: User

Use Cases: Add Meeting, View Meetings, Delete Meeting, Check Reminders, Exit

7.2 Activity Diagram (Workflow of Adding a Meeting)

Start → Display Menu → Choose 1 → Input Title → Input Date → Input Time → Validate → Save → Success/Fail → Back to Menu

7.3 Sequence Diagram (Check Upcoming Meetings)

User → main() → upcoming() → Loop through meetings → Calculate delta → Print if within 24h → Return

7.4 Component Diagram

Components: Menu Module, Add Module, View Module, Delete Module, Reminder Module, File Handler

7.5 ER Diagram

Entity: Meeting

Attributes: title (string), time (datetime)

Storage: Comma-separated lines in bucket.txt

---

8. Design Decisions & Rationale

Decision	Options Considered	Chosen Option	Reason
Storage	Text file vs JSON vs SQLite	Text file	Zero dependency, simple parsing
Date/Time Input	Single string vs separate	Separate fields	Easier validation and user-friendliness
Sorting	Manual sort vs sorted()	Lambda with sorted()	Clean, readable, efficient
Reminder Logic	Exact time vs range	Next 24 hours	Practical real-world use

Error Handling	Try-except vs manual checks	Try-except	Catches invalid date entries gracefully

---

9. Implementation Details

Key Functions

- file_load() – Loads meetings from data.txt on startup
- input_save() – Saves current list to file after every change
- add() – Takes structured input and validates date/time
- show() – Displays meetings sorted chronologically
- upcoming() – Calculates time difference using datetime.timedelta

Sample File Format (bucket.txt)

text

```
Team Standup,2025-11-24 10:00
Client Demo,2025-11-25 15:30
Project Review,2025-11-23 14:00
```

No external libraries required → Pure Python

---

10. Screenshots / Results

```
MEETING SCHEDULER
1.Add a meeting
2.View all meetings
3.Delete a meeting
4.Check reminders
5.Exit
Enter your choice (1-5): 1

ADD NEW MEETING
Enter meeting Title: job meeting

  ENTER DATE
Year(YYYY): 2025
Month(MM): 12
Day(DD): 23

  ENTER TIME
Hour (0-23): 14
Minute (0-59): 0
Meeting saved!
Press Enter to continue...

MEETING SCHEDULER
1.Add a meeting
2.View all meetings
3.Delete a meeting
4.Check reminders
5.Exit
Enter your choice (1-5): 2

ALL MEETINGS
1. walk
Date: 2025-11-24
Time: 06:24

2. job
Date: 2025-12-12
Time: 12:12

3. job meeting
Date: 2025-12-23
Time: 14:00

Press Enter to continue...
```

```
MEETING SCHEDULER
1.Add a meeting
2.View all meetings
3.Delete a meeting
4.Check reminders
5.Exit
Enter your choice (1-5): 3

DELETE MEETING
1. walk - 2025-11-24 06:24
2. job meeting - 2025-12-23 14:00
Enter meeting number to delete (0 to cancel): 2
Meeting deleted!
Press Enter to continue...

MEETING SCHEDULER
1.Add a meeting
2.View all meetings
3.Delete a meeting
4.Check reminders
5.Exit
Enter your choice (1-5): 4

 UPCOMING MEETINGS (NEXT 24 HOURS)

walk
At: 2025-11-24 06:24
In: 6h 59m
Press Enter to continue...

MEETING SCHEDULER
1.Add a meeting
2.View all meetings
3.Delete a meeting
4.Check reminders
5.Exit
Enter your choice (1-5): 5
Goodbye!
```

1. **Main Menu**
2. **Adding a new meeting**
3. **Invalid past date error**
4. **Viewing all meetings (sorted)**
5. **Deleting a meeting**
6. **Upcoming meetings reminder**
7. **Empty state message**
8. **bucket.txt file content**

11. Testing Approach

Test Case	Input	Expected Output	Status
Add valid meeting	Title + valid future date/time	"Meeting saved!"	Passed
Add invalid date	Month=13	"Invalid date or time!"	Passed
View meetings	Multiple entries	Sorted by date/time	Passed
Delete valid meeting	Correct number	"Meeting deleted!"	Passed
Check upcoming (within 24h)	Meeting in next 10 hours	Shows with remaining time	Passed
No upcoming meetings	All meetings >24h away	"No upcoming meetings"	Passed
Data persistence	Restart program	Previously added meetings still present	Passed

---

12. Challenges Faced

Challenge	Solution Implemented
Parsing date/time from file	Used strict strptime with exact format
Sorting meetings chronologically	Used sorted() with lambda key on time
Calculating time remaining	Used timedelta and integer division

Handling invalid user inputs	Try-except + input validation
Data loss on crash/corruption	Save after every change (immediate persistence)

---

13. Learnings & Key Takeaways

- Deep understanding of Python's datetime module
- Importance of persistent storage even in small apps
- Real-world use of file I/O for configuration/data
- Clean code organization improves maintainability
- User experience matters even in console apps (clear prompts, feedback)

---

14. Future Enhancements

- GUI using Tkinter or PyQt
- Sound/email notification when a meeting is due
- Recurring meetings support
- Export to CSV / Google Calendar sync
- Search and edit meeting functionality
- Reminder daemon (runs in background)
- Support for multiple users / categories

---

15. References

1. Python Documentation – datetime module https://docs.python.org/3/library/datetime.html
2. GeeksforGeeks – Python File Handling

3. **Real Python – Working with Dates and Times**
4. **“Automate the Boring Stuff with Python” by Al Sweigart (Chapter on Files & CSV)**